

Hash Tables & Binary Search Trees

Hash Tables

A Hash Table `ItemHashMap` is used in the project to store items based on their IDs. The class implements a hash table that serves as the main repository for items. The hash table uses universal hashing and a Rabin-Karp rolling hash to generate hash codes for the items based on their IDs. Upon creation with the `fromTable` method, the table will also print the number of buckets with 0 entries, 1 entry, 2 entries, etc. for both the custom hash function and the built-in `String.hashCode()` method. This allows for a comparison of the distribution of items in the hash table using the custom hash function versus the built-in hash function. The `ItemHashMap` class also implements the `'ItemRepo'` interface, which abstracts away all functionality of the hash table except for retrieval of an item by ID, retrieving all items, and checking the size of the hash table. This allows for the hash table to be easily swapped out for a different implementation if needed in the future, without affecting the rest of the codebase. The hash map is used very frequently throughout the program, as it makes it simple to retrieve an item entity with just the ID.

- Implementation Details
 - A custom hash key was used instead of the built-in `String.hashCode()` method to allow for custom hashing functionality.

Binary Search Trees

While a pure Binary Search Tree is not used in the project, a Red-Black Tree is used in the `PriceTree` class to store items based on their price. The class extends a `TreeMap` in Java, which is implemented as a Red-Black Tree, a self-balancing binary search tree. This allows for efficient searching of items within a certain price range using the `subMap` method of the `TreeMap`. The `PriceTree` class uses this functionality to implement the price filter in the search engine.

- Implementation Details
 - The Red-Black Tree ensures that the tree remains balanced, providing $O(\log n)$ time complexity for insertion, deletion, and lookup operations.
 - Under the current implementation, item price's must be unique, as the price is used as the key in the tree. If duplicate prices were allowed, a list of item indices would need to be stored at each node instead of a single item index. This may be changed in a future release, but for the current implementation, it is assumed that all item prices are unique. In the data set used for the project, all item prices are unique, so this does not cause any issues.
- Why it was chosen
 - The `TreeMap` class in Java already implements a Red-Black Tree and provides a `subMap` method that allows for efficient searching of items within a certain price range. This made it a convenient choice for implementing the price filter. While it is certainly possible to make a custom tree class to handle the filtering, the built-in `TreeMap` provides a lot of functionality out of the box that makes it simple to implement the price filter, while ensuring optimal performance.
 - The Red-Black tree choice is because it is self-balancing, ensuring a maximum search time of $O(\log n)$. This is important for the price filter, as it allows for efficient searching of items within a certain price range, even as the number of items increases.